Escola de Engenharia
Programa de Pós-Graduação em Engenharia Elétrica

EEE889 - Eletromagnetismo Computacional

Raphael Henrique Braga Leivas

LISTA 4

Belo Horizonte
2 de maio de 2026

SUMÁRIO

1 EXERCÍCIO 5.7 . . . . . **3**
2 EXERCÍCIO 6.1 . . . . . **5**
2.1 Item (a) 5
2.2 Item (b) 6
2.3 Item (c) 6

ANEXO A CÓDIGO EXERCÍCIO 5.7 . . . . . **9**
ANEXO B CÓDIGO EXERCÍCIO 6.1 . . . . . **11**

1 EXERCÍCIO 5.7

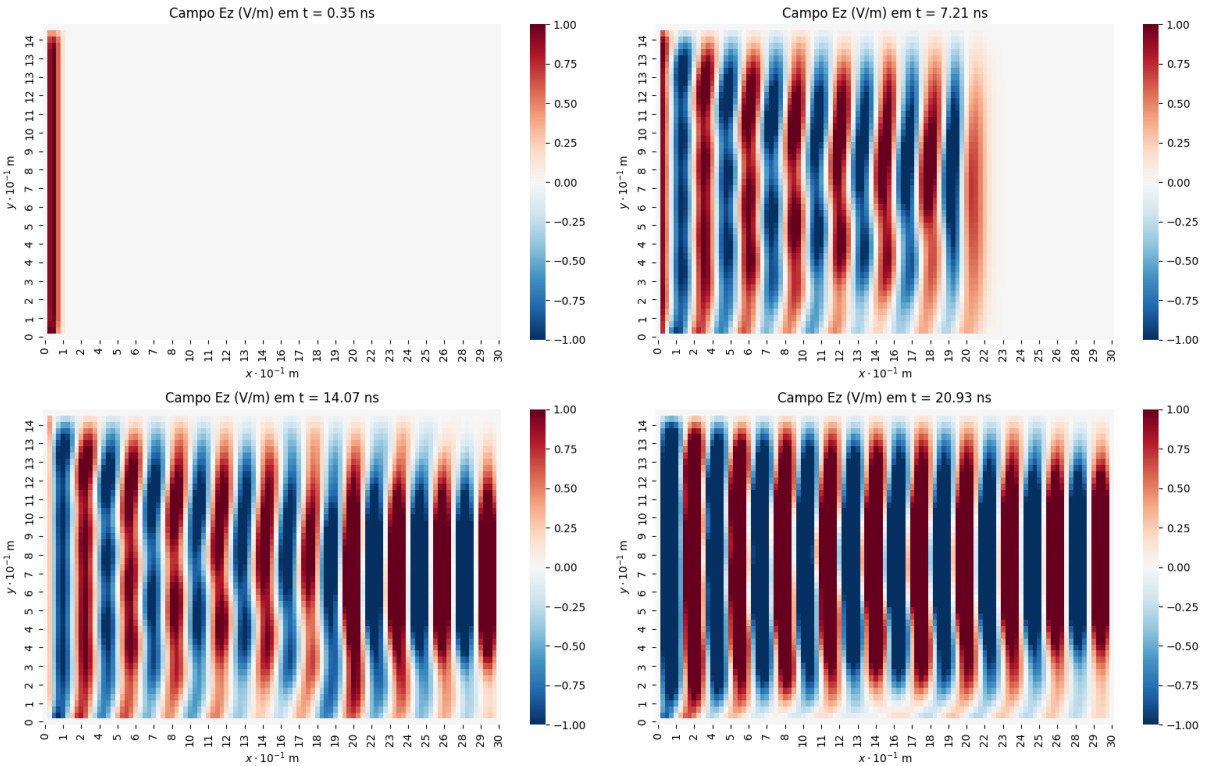
Todo o código usado nesse exercício encontra-se no Anexo A.

O primeiro passo é obter uma simulação estável na geometria proposta pelo problema. Considerando uma fonte de onda planar senoidal com $f = 1$ GHz no espaço livre, temos $\lambda = c/f = 0.3$ m. Usando os seguintes parâmetros de discretização:

- $N_x = 500 \quad , \quad N_y = 100$
- $\Delta y = \Delta x = 0.1 \cdot \lambda = 0.03$ m
- $\Delta t = \frac{0.99}{c\sqrt{(\Delta x)^{-2}+(\Delta y)^{-2}}} = 70$ ps (condição CFL 2D)
- $N_t = 500$

obtemos a seguinte propagação da onda planar a partir da extremidade esquerda do grid, exibida na Figura 1.

Figura 1 – Propagação da onda planar senoidal a partir da extremidade esquerda.



Note que ao atingir a borda direita do grid temos reflexão por ser um PEC, obtendo assim um padrão de interferência.

Variando o valor de Δt, e usando uma ponta de prova no centro do grid, podemos usar o valor máximo medido para aferir a estabilidade do método em função de Δt, como mostra a Tabela 1. Vemos que o método se torna instável quando a condição de CFL não é respeitada.

COmo deduzir a CFL nesse caso a partir de Von Neumann

Tabela 1 – Valor máximo medido no centro do grid em função de Δt.

Δt	E_z Máximo (V/m)
$0.50 \quad / \quad c\sqrt{(\Delta x)^{-2} + (\Delta y)^{-2}}$	1.18
$0.90 \quad / \quad c\sqrt{(\Delta x)^{-2} + (\Delta y)^{-2}}$	1.25
$0.98 \quad / \quad c\sqrt{(\Delta x)^{-2} + (\Delta y)^{-2}}$	1.25
$0.99 \quad / \quad c\sqrt{(\Delta x)^{-2} + (\Delta y)^{-2}}$	1.24
$1.00 \quad / \quad c\sqrt{(\Delta x)^{-2} + (\Delta y)^{-2}}$	1.25
$1.01 \quad / \quad c\sqrt{(\Delta x)^{-2} + (\Delta y)^{-2}}$	$1.13 \cdot 10^{54}$
$1.10 \quad / \quad c\sqrt{(\Delta x)^{-2} + (\Delta y)^{-2}}$	$1.016 \cdot 10^{184}$

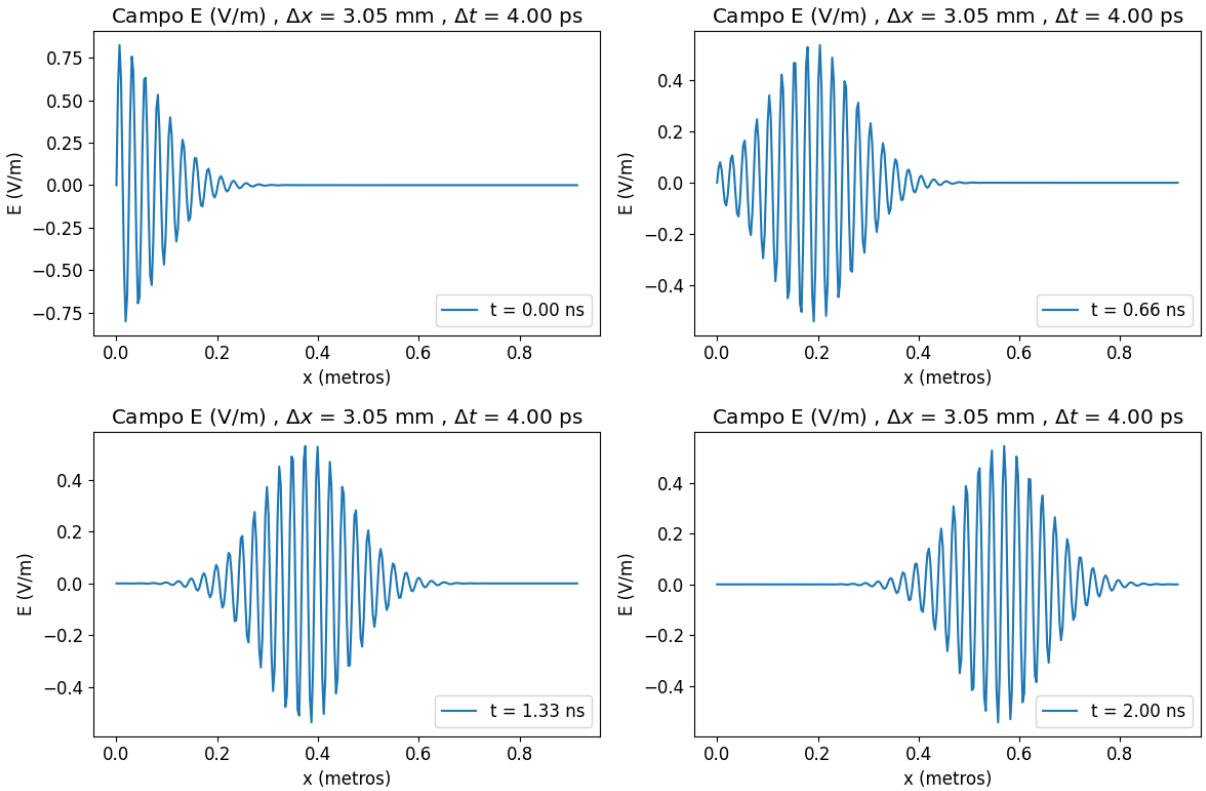
2 EXERCÍCIO 6.1

Todo o código usado nesse exercício encontra-se no Anexo B.

2.1 *Item (a)*

Simulando a geometria do problema com o FDTD em 1 dimensão, temos a progragação da onda exibida na Figura 2.

Figura 2 – Propagação da onda até $t = 2$ ns.



Fixando o pico do pulso gaussiano e coletando a sua posição ao longo do tempo, obtemos o gráfico da Figura 3. A inclinação dessa reta é a velocidade de fase v_p da onda, obtida via regressão linear.

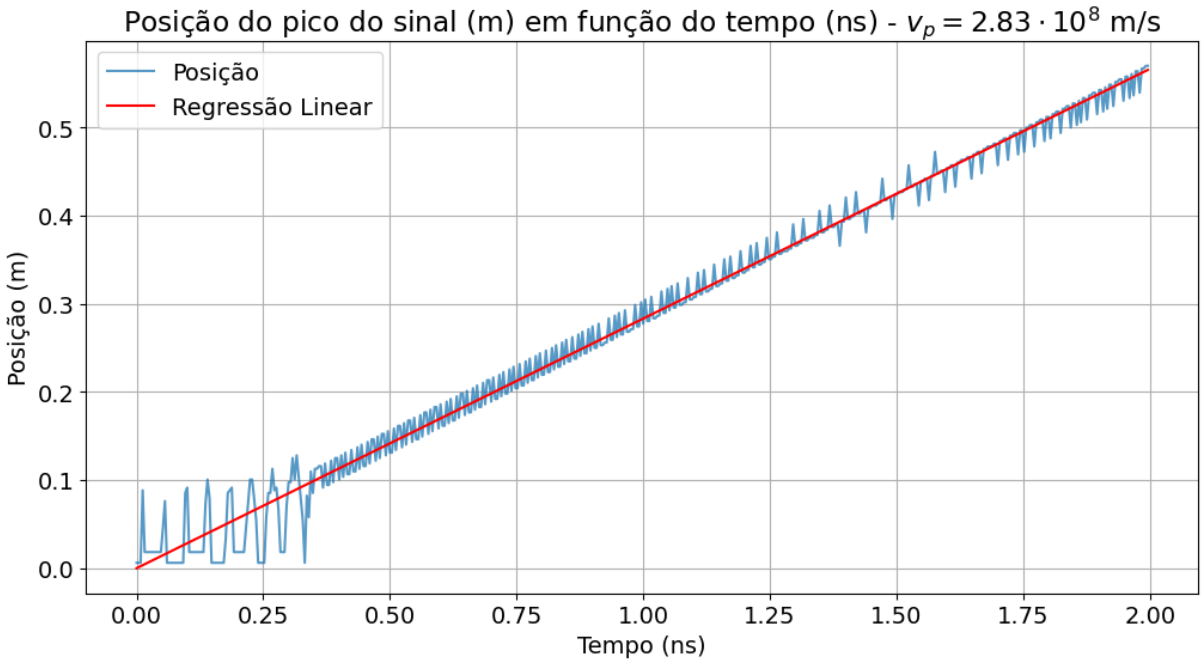
Variando os parâmetros de discretização, vemos que velocidade de fase numérica se aproxima da velocidade analítica $3 \cdot 10^8$ m/s, como mostra a Tabela 2.

Tabela 2 – Velocidade de fase numérica obtida em função da discretização.

Δt (ps)	Δx (mm)	v_p ($10^8 \cdot$ m/s)
4	3.05	2.83
3	2.29	2.86
2	1.52	2.99

Como calcular a velocidade de grupo

Figura 3 – Posição do pico do pulso gaussiano em função do tempo simulado.



2.2 *Item (b)*

Simulando o novo pulso, obtemos a progação exibida na Figura 4. Usando o mesmo método descrito no item (a), obtemos as seguintes velocidades de fase em função da descritização, exibida na Tabela 3.

Tabela 3 – Velocidade de fase numérica obtida em função da discretização.

Δt (ps)	Δx (mm)	v_p ($10^8 \cdot$ m/s)
5	3.05	3.03
3	2.29	3.02
2	1.52	3.02

2.3 *Item (c)*

Com a nova fonte do item (c), obtemos a progação exibida na Figura 5. Usando o mesmo método descrito no item (a), obtemos as seguintes velocidades de fase em função da descritização, exibida na Tabela 3. A posição do pico do sinal em função do tempo, bem como a v_p calculada a partir da regressão linear estão exibidos na Figura 6. Com a discretização sugerida no enunciado, que é bastante refinada, já obtemos um valor igual ao analítico para a velocidade de fase.

Figura 4 – Propagação da onda até $t = 4$ ns.

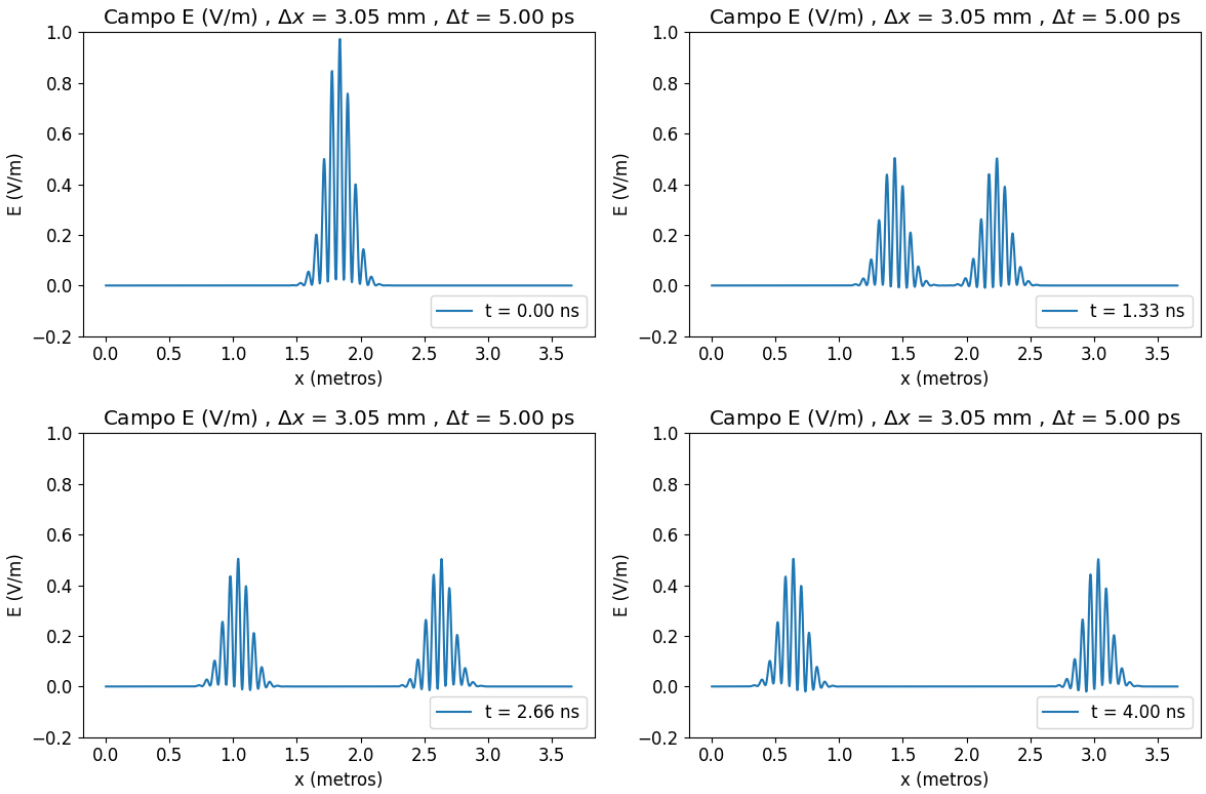


Figura 5 – Propagação da onda até $t = 2$ ns.

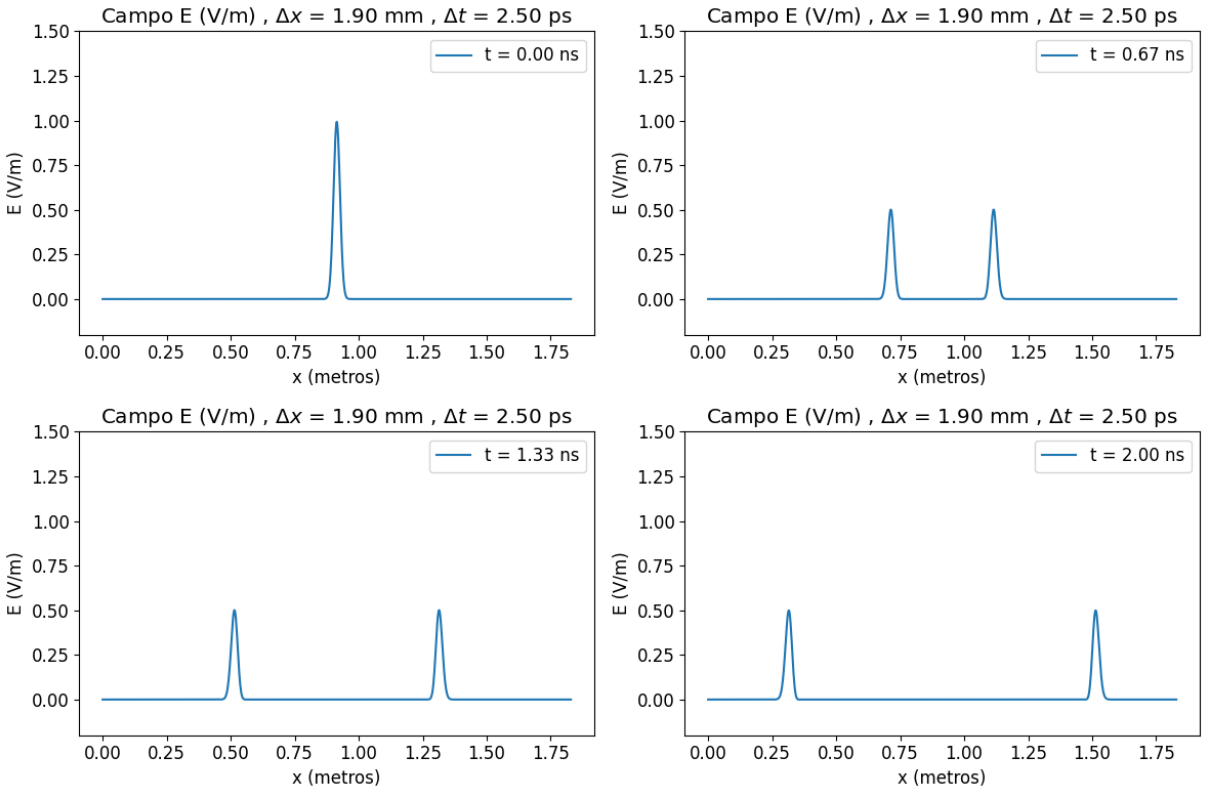
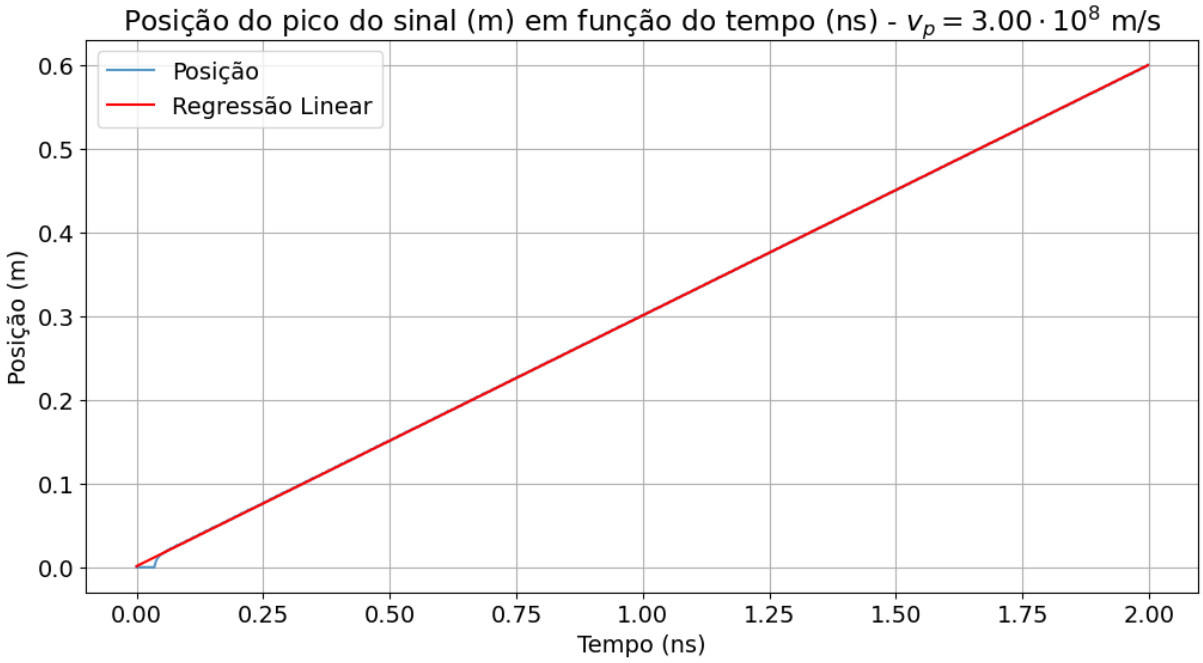


Figura 6 – Posição do pico do pulso gaussiano em função do tempo simulado.



ANEXO A – CÓDIGO EXERCÍCIO 5.7

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.fft import fft, fftfreq

# --- parâmetros físicos ---
eps = 8.854187817e-12  # Vacuum permittivity (F/m)
mu = 4 * np.pi * 1e-7  # Vacuum permeability (H/m)
vp = 1 / np.sqrt(eps * mu)
f = 1e9

# --- domínio ---
min_lambda = vp / f  # comprimento de onda mínimo para 1 GHz = 3e-1 m
Nx = 100
Ny = 50
dx = dy = 0.1 * min_lambda
dt_list = [0.5, 0.9, 0.98, 0.99, 1, 1.01, 1.1] / (vp * np.sqrt((1/dx**2) + (1/dy**2)))
# dt_list = [0.99] / (vp * np.sqrt((1/dx**2) + (1/dy**2)))
max_value_per_dt = []

for dt in dt_list:
    Nt = 500
    L = Nx * dx
    T = Nt * dt

    grid_x_Ez = np.linspace(0, L, Nx + 1)
    grid_y_Ez = np.linspace(0, L, Ny + 1)
    grid_t_Ez = np.linspace(0, T, Nt + 1)

    grid_x_Hx = np.linspace(0, L, Nx + 1)
    grid_y_Hx = np.linspace(dy/2, L - dy/2, Ny)
    grid_t_Hx = np.linspace(dt/2, T - dt/2, Nt)

    grid_x_Hy = np.linspace(dx/2, L - dx/2, Nx)
    grid_y_Hy = np.linspace(0, L, Ny + 1)
    grid_t_Hy = np.linspace(dt/2, T - dt/2, Nt)

    # --- campos ---
    Ez_curr = Ez_next = np.zeros((grid_x_Ez.shape[0], grid_y_Ez.shape[0]))
    Hx_curr = Hx_next = np.zeros((grid_x_Hx.shape[0], grid_y_Hx.shape[0]))
    Hy_curr = Hy_next = np.zeros((grid_x_Hy.shape[0], grid_y_Hy.shape[0]))

    # valores do tempo para salvar os snapshots e montar os heatmaps
    n_snapshots = np.linspace(5, Nt - 5, 6, dtype=int)
    Ez_snapshots = []

    # ponta de prova no centro
    probe = []
    cx, cy = Nx // 2, Ny // 2

    for n in range(Nt-1):
        # fonte onda planar
        source = np.exp(-((n * dt - 1.8e-9)**2) / (0.6e-9)**2)
        source_senoidal = np.sin(2 * np.pi * f * n * dt)

        # aplica em todos os Y em X = 0 (left boundary plane wave)
        Ez_curr[0, :] = source_senoidal
```

```
    Hx_next[:, :] = Hx_curr[:, :] - (dt/(mu*dy)) * (
        Ez_curr[:, 1:] - Ez_curr[:, :-1]
    )

    Hy_next[:, :] = Hy_curr[:, :] + (dt/(mu*dx)) * (
        Ez_curr[1:, :] - Ez_curr[:-1, :]
    )

    Ez_next[1:-1, 1:-1] = Ez_curr[1:-1, 1:-1] + (dt/eps) * (
        (Hy_next[1:, 1:-1] - Hy_next[:-1, 1:-1]) / dx -
        (Hx_next[1:-1, 1:] - Hx_next[1:-1, :-1]) / dy
    )

    # PEC an extremidade direita
    Ez_next[-1, :] = 0

    # periodic boundary condition
    Ez_next[:, 0] = Ez_next[:, Ny]
    Hx_next[:, 0] = Hx_next[:, Ny - 1]

    # atualiza os campos para o próximo passo
    Ez_curr = Ez_next.copy()
    Hx_curr = Hx_next.copy()
    Hy_curr = Hy_next.copy()

    # salva na ponta de prova
    probe.append(Ez_curr[cx, cy])

    # salva para os snapshots
    if n in n_snapshots:
        Ez_snapshots.append(Ez_curr.copy())

max_value_per_dt.append(np.max(probe))
```

ANEXO B – CÓDIGO EXERCÍCIO 6.1

```
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams.update({'font.size': 12})

# Constantes
eps = 8.854187817e-12  # Vacuum permittivity (F/m)
mu = 4 * np.pi * 1e-7  # Vacuum permeability (H/m)
vp = 1 / np.sqrt(eps * mu)
FEET_TO_M = 3.281

T = 2e-9 # tempo de simulação
L = 3 / FEET_TO_M # distancia para simular

dx = 0.01 / FEET_TO_M
dt = 4e-12

# check CFL
if dt > dx / vp:
    raise ValueError(f"CFL␣not␣met:␣{dt}␣>␣{dx␣/␣vp}")

nt = int(T / dt)
nx = int(L / dx)

# grids: armazenam os nós
# OBS: nx + 1 nós implicam em nx intervalos
grid_xE = np.linspace(0, L, nx + 1) # xgrid para o Ey ; +1 pois sao nx intervalos de delta
    x
grid_tE = np.linspace(0, T, nt + 1) # tgrid para o Ey ; +1 pois sao nt intervalos de delta
    t

grid_xH = np.linspace(dx / 2, L - dx / 2, nx) # xgrid para o Hz ; temos nx - 1 intervalos
    de delta x
grid_tH = np.linspace(dt / 2, T - dt / 2, nt) # tgrid para o Hz ; temos nt - 1 intervalos
    de delta t

E_curr = np.exp(-(8 * grid_xE)**2) * np.sin(250 * grid_xE) # condicao inicial
E_next = np.zeros(grid_xE.shape[0])

H_curr = - (dt / (mu * dx)) * (E_curr[1:] - E_curr[:-1]) # estende a condição inicial para
    o H
H_next = np.zeros(grid_xH.shape[0])

# snapshots para as fotos
snapshot_times = np.linspace(0, nt - 1, 4, dtype=int)
E_snapshots = []

# monitora o pico do pulso
posicao_pico = []

# marcha no tempo - calculamos o next a cada interação
for n in range(nt):
    # equações do FDTD 1D
    H_next = H_curr - dt / (mu * dx) * (E_curr[1:] - E_curr[:-1])
    E_next[1:-1] = E_curr[1:-1] - dt / (eps * dx) * (H_next[1:] - H_next[:-1])

    # condição de contorno: PEC nas bordas
    E_next[0] = E_next[-1] = 0
```

```
    # tira as fotos
    if n in snapshot_times:
        E_snapshots.append(E_next.copy())

    # salva a posicao atual do pico gaussiano
    posicao_pico.append(grid_xE[np.argmax(E_next)])

    # atualiza o corrente para o proximo passo
    E_curr = E_next
    H_curr = H_next

# plota a posicao do pico em função do tempo
slope, intercept = np.polyfit(grid_tE[:-1], posicao_pico, 1)

plt.figure(figsize=(12, 6))  # 10 inches wide, 5 inches tall
plt.rcParams.update({'font.size': 14})

plt.plot(grid_tE[:-1] * 1e9, posicao_pico, label="Posição", alpha=0.75)
plt.plot(grid_tE[:-1] * 1e9, slope * grid_tE[:-1], color="red", label="Regressão Linear")
plt.title(fr"Posição do pico do sinal (m) em função do tempo (ns) - $v_p = {slope / 1e8:.2
    f} \cdot 10^{8}$ m/s")
plt.xlabel("Tempo (ns)")
plt.ylabel("Posição (m)")
plt.grid()
plt.legend()
```